Introdução à Computação
merge sort

Alexandre Rademaker

Ordenação (sort) I

Dado uma lista de valores, possívelmente desordenada, queremos escrever um algorítmo que ordena esta lista, segundo algum critério.

Por exemplo, para a lista

$$[6\ 3\ 8\ 5\ 2\ 7\ 4\ 1]$$

queremos produzir

$$[1\ 2\ 3\ 4\ 5\ 6\ 7\ 8]$$

Ordenação (sort) II

Vamos considerar dois algorítmos para esta tarefa:

- selection sort
- bubble sort
- merge sort

Merge sort I

O merge sort é um algorítmo recursivo para ordenação.

Composto de duas etapas: divisão e merge. Na divisão, o array inicial é dividido em duas metades e o algorítmo se chama para ordenar as metades.

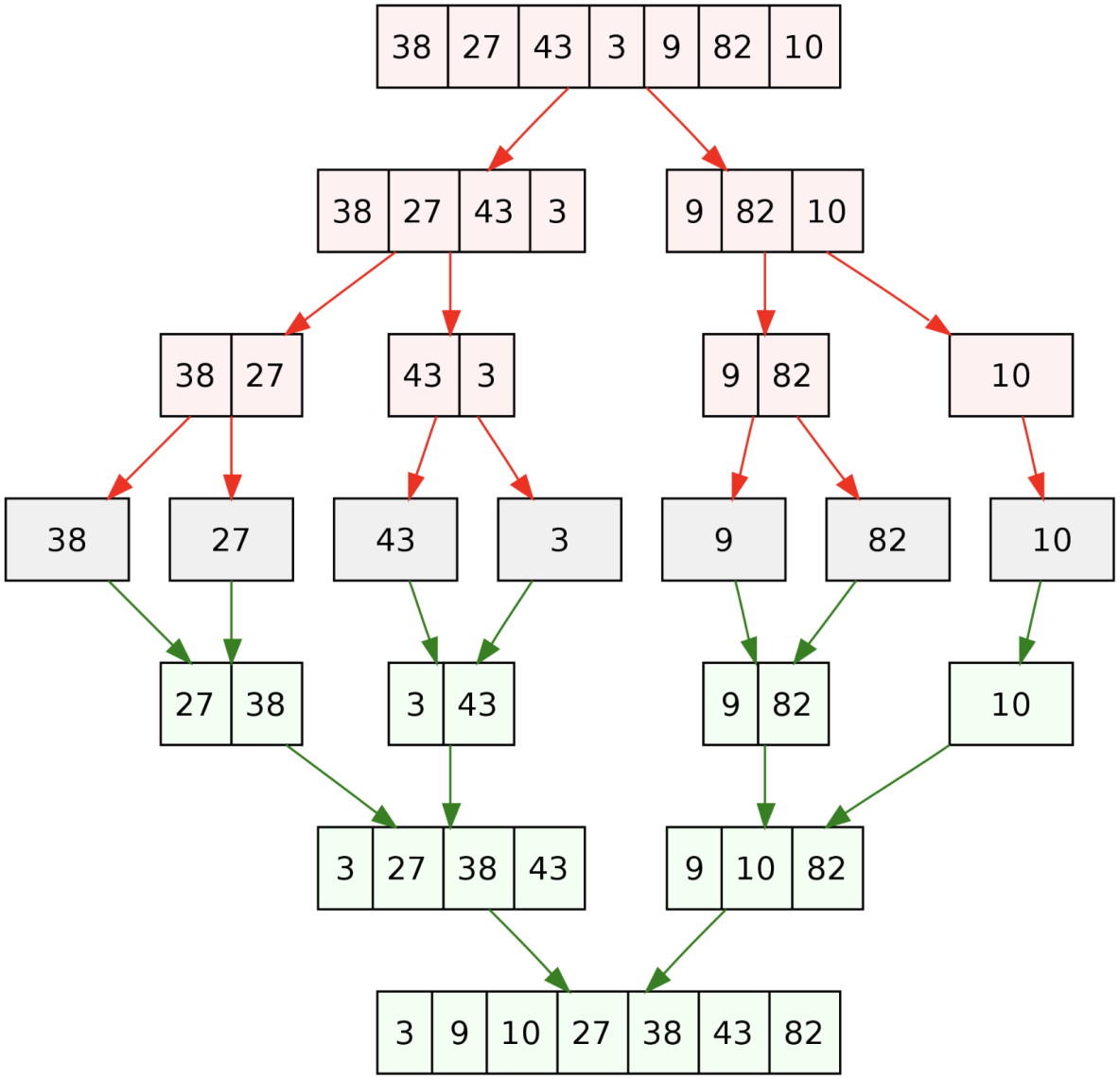
No merge, dois arrays ordenados são linearmente juntados em um único array também ordenado.

Merge sort II

```
function merge_sort(a[1...n])
  if n > 1:
   x = merge_sort(a[1...n/2])
   y = merge_sort(a[n/2+1...n])
   return merge(x,y)
  else:
   return a

function merge(x[1...k],y[1...l])
 if k == 0: return y[1...l]
 if l == 0: return x[1...k]
 if x[1] < y[1]:
  return x[1] . merge(x[2...k,y[1...l])
 else:
  return y[1] . merge(x[1...k,y[2...l])
```

Merge sort III



Merge sort IV

Juntar duas listas ordenadas pode ser feito em tempo $O(n)$ onde n é o tamanho da lista resultante.

A quantidade de passos é definida de forma recursiva, n elementos geram duas ordenações de $n/2$ e mais um custo linear para o merge.

$$T(n) = 2T(\frac{n}{2}) + O(n)$$

Complexidade é $O(n \log n)$. Por que? Pelo teorema mestre.

A função merge I

```
2  4  5  7  |  0  1  3  6

2  4  5  7  |  0  1  3  6
^            ^

2  4  5  7  |     1  3  6
^              ^
0

2  4  5  7  |        3  6
^                 ^
0  1

4  5  7  |        3  6
^              ^
0  1  2
```

A função merge II

src/merge.c

```
1  void merge (int ns[], int l, int m, int u) {
2    int t[u - l + 1];
3    for (int k = 0, a = l, b = m + 1;
4         k < (u - l + 1);
5         k++) {
6      t[k] =          a > m ? ns[b++] // empty left
7           :          b > u ? ns[a++] // empty right
8           : ns[a] < ns[b] ? ns[a++] // take left
9           : ns[b++];                // take right
10   }
11
12   for (int i = l; i <= u; i++)
13    ns[i] = t[i - l];
14 }
```

A função merge_sort I

```
5 2 7 4 | 1 6 3 0

5 2 | 7 4

5 | 2

(podemos juntar)

2 5
```

Recursivamente divide o array passado em duas metades.

A função merge_sort II

src/merge.c

```
1 void merge_sort (int a[], int i, int j) {
2   if ((j - i) == 0)
3     return;
4
5   int m = (i + j) / 2;
6   merge_sort(a, i, m);
7   merge_sort(a, m + 1, j);
8   merge(a, i, m, j);
9 }
```

A função `merge_sort` III

O que fizemos diferente do pseudo-código?

Ao invés de 'retornar' arrays, alteramos os arrays passados.

O merge poderia ser feito 'in place', isto é, sem uso de memória extra? Se array de entrada tivesse 10^{200} posições?

Conclusão I

Existem outros algorítmos de ordenação.

Um site interessante de algorítmos e implementações em várias linguagens de programação é o http://rosettacode.org.

Podemos visualizar e comparar o desempenho dos algorítmos.

Linguages de programação variam em diversos aspectos, como o foco em abstrações de alto nível versus controle preciso de recursos.

Conclusão II

```haskell
merge :: Ord a => [a] -> [a] -> [a]
merge [] ys = ys
merge xs [] = xs
merge xs@(x:xt) ys@(y:yt)
 | x <= y     = x : merge xt ys
 | otherwise = y : merge xs yt

split :: [a] -> ([a], [a])
split []   = ([],[])
split [x] = ([x],[])
split (x:y:zs) = (x:xs,y:ys)
 where (xs,ys) = split zs

-- >>> split [1,2,3]
-- ([1,3],[2])

mergeSort :: Ord a => [a] -> [a]
mergeSort []   = []
mergeSort [x] = [x]
mergeSort xs   = merge (mergeSort as) (mergeSort bs)
 where (as,bs) = split xs

-- >>> mergeSort [1,2,4,3]
-- [1,2,3,4]
```

Conclusão III

Programadores Haskell adoram abstração (e tipos, funções, variáveis imutáveis, pattern matching . . .).

Mas o código Haskell faz exatamente a mesma coisa que o código C?

Programadores C adoram controle preciso de recursos!

Conclusão IV

"The real problem is that programmers have spent far too much time worrying about efficiency in the wrong places and at the wrong times; premature optimization is the root of all evil (or at least most of it) in programming."

The Art of Computer Programming, Donald Knuth